

# Conventional Commits — Гайд по написанию коммитов

Стандарт форматирования коммитов для TimeToEvent.

## Зачем нужен стандарт?

- **Автоматическая генерация CHANGELOG** — типы коммитов используются для создания истории изменений
- **Понятная история** — сразу видно, что изменилось
- **Упрощённый code review** — по типу коммита понятно, что проверять
- **Автоматизация релизов** — semantic-release использует типы для определения версии

## Формат коммита

```
<type>(<scope>): <description>
```

```
[optional body]
```

```
[optional footer(s)]
```

## Примеры

```
feat(events): добавить форму создания события
```

- Добавлена валидация через react-hook-form + zod
- Реализован выбор даты и времени
- Добавлен ColorPicker для выбора цвета

```
Closes #123
```

`fix(sync)`: исправить race condition в broadcast

При одновременном изменении события на двух устройствах возникал конфликт. Добавлена блокировка на уровне БД.

BREAKING CHANGE: изменён формат `sync_log`

## Типы коммитов

| Тип      | Описание                                   | Пример                                                            |
|----------|--------------------------------------------|-------------------------------------------------------------------|
| feat     | Новая фича                                 | <code>feat(events)</code> : добавить поиск по событиям            |
| fix      | Исправление бага                           | <code>fix(sync)</code> : исправить утечку памяти в WebSocket      |
| docs     | Изменения в документации                   | <code>docs(readme)</code> : обновить инструкции по установке      |
| style    | Форматирование, точки с запятой            | <code>style</code> : исправить отступы в <code>eventsStore</code> |
| refactor | Рефакторинг без изменения функциональности | <code>refactor(crypto)</code> : вынести HKDF в отдельную функцию  |
| test     | Добавление/изменение тестов                | <code>test(aes)</code> : добавить тесты для шифрования            |
| chore    | Изменения в сборке, зависимостях, CI       | <code>chore(deps)</code> : обновить <code>tokio</code> до 1.52.3  |
| perf     | Улучшение производительности               | <code>perf(events)</code> : добавить индексы SQLite               |
| ci       | Изменения в CI/CD                          | <code>ci</code> : добавить GitHub Actions для тестов              |



## Score (область изменений)

Score указывает, какая часть проекта изменена. Необязательный, но рекомендуемый.

### Примеры score

- `events` — события и таймеры
- `sync` — синхронизация
- `crypto` — криптография
- `ui` — интерфейс
- `db` — база данных
- `deps` — зависимости
- `readme` — [README.md](#)
- `config` — конфигурация

### Когда не указывать score

Если изменение затрагивает несколько областей или слишком мелкое:

`style:` исправить форматирование

`chore:` обновить зависимости



## Описание (description)

### Правила

1. **Максимум 72 символа** — чтобы помещалось в `git log`
2. **Начинается с заглавной буквы** — `feat: Добавить...` НЕ `feat: добавь...`
3. **Без точки в конце** — `feat: Добавить поиск` НЕ `feat: Добавить поиск.`
4. **Используй повелительное наклонение** — `добавить` НЕ `добавил` ИЛИ `добавляет`

### Примеры

✅ **Правильно:**

feat(events): добавить поиск по событиям  
fix(sync): исправить race condition в broadcast  
docs(readme): обновить инструкции по установке

## ❌ Неправильно:

feat: добавил поиск  
fix: исправляю баг  
docs: обновление readme



## Body (тело коммита)

Необязательная часть. Используется для подробного описания изменений.

## Когда использовать

- Сложные изменения, требующие объяснения
- Breaking changes
- Изменения, влияющие на несколько модулей

## Формат

feat(events): добавить поиск по событиям

- Добавлен debounce 300ms для оптимизации
- Поиск работает по названию и описанию
- Добавлен EmptyState для пустых результатов

Closes #123



## Footer (подвал коммита)

Необязательная часть. Используется для ссылок на issues и breaking changes.

# Breaking changes

`refactor(crypto)`: изменить формат ключей

BREAKING CHANGE: ключи теперь хранятся в base64 вместо hex.

При обновлении потребуется регенерация ключей.

## Ссылки на issues

`fix(sync)`: исправить утечку памяти

Closes #123

Related to #456



## Автоматизация

### Commitlint (проверка формата)

```
npm install --save-dev @commitlint/cli @commitlint/config-conventional
```

Создай `commitlint.config.js`:

```
module.exports = {  
  extends: ['@commitlint/config-conventional'],  
  rules: {  
    'type-enum': [2, 'always', [  
      'feat', 'fix', 'docs', 'style', 'refactor', 'test', 'chore', 'perf', 'ci'  
    ]],  
    'subject-max-length': [2, 'always', 72],  
  }  
};
```

# Husky (git hooks)

```
npm install --save-dev husky
npm husky install
npm husky add .husky/commit-msg 'npx --no -- commitlint --edit ${1}'
```

## Как типы влияют на CHANGELOG

| Тип      | Секция в CHANGELOG         |
|----------|----------------------------|
| feat     | 🌟 Features                 |
| fix      | 🐛 Bug Fixes                |
| perf     | ⚡ Performance Improvements |
| refactor | 🔧 Code Refactoring         |
| test     | 🧪 Tests                    |
| docs     | 📖 Documentation            |
| style    | 🎨 Styles                   |
| chore    | 🔨 Chores                   |
| ci       | CI                         |

## Best practices

### 1. Один коммит = одно логическое изменение

Плохо:

```
feat: добавить поиск, исправить баг в синхронизации, обновить зависимости
```

✅ Хорошо:

```
feat(events): добавить поиск по событиям  
fix(sync): исправить race condition  
chore(deps): обновить tokio до 1.52.3
```

## 2. Коммить часто, но осмысленно

Не копи код на неделю в один коммит. Делай небольшие коммиты по мере выполнения задач.

## 3. Не коммить отладочный код

```
# Плохо  
console.log('DEBUG: ', data);  
// TODO: убрать это
```

## 4. Использовать pre-commit хуки

Автоматическая проверка формата перед коммитом экономит время.

## Полезные ссылки

- [Conventional Commits Specification](#)
- [Angular Commit Guidelines](#)
- [Commitlint](#)
- [Husky](#)